06-01 Blockstore 구현 분석

개요

Blockstore는 Solana의 핵심 데이터 저장 시스템으로, RocksDB를 기반으로 블록체인 데이터를 영구적으로 저장하고 관리합니다. 이 시스템은 shred(블록 조각), 메타데이터, 트랜잭션 상태 등 다양한 유형의 데이터를 효율적으로 저장하고 검색할 수 있도록 설계되었습니다.

핵심 구조체

Blockstore 구조체

```
pub  struct Blockstore {
ledger_path: PathBuf,
db: Arc<Database>,
// Column Families
meta_cf: LedgerColumn<cf::SlotMeta>,
dead_slots_cf: LedgerColumn<cf::DeadSlots>,
duplicate_slots_cf: LedgerColumn<cf::DuplicateSlots>,
roots_cf: LedgerColumn<cf::Root>,
erasure_meta_cf: LedgerColumn<cf::ErasureMeta>,
orphans_cf: LedgerColumn<cf::Orphans>,
index_cf: LedgerColumn<cf::Index>,
data_shred_cf: LedgerColumn<cf::ShredData>,
```

```
code_shred_cf: LedgerColumn<cf::ShredCode>,

transaction_status_cf: LedgerColumn<cf::TransactionStatus>,

address_signatures_cf: LedgerColumn<cf::AddressSignatures>,

transaction_memos_cf: LedgerColumn<cf::TransactionMemos>,

transaction_status_index_cf: LedgerColumn<cf::TransactionStatusIndex>,

rewards_cf: LedgerColumn<cf::Rewards>,

blocktime_cf: LedgerColumn<cf::Blocktime>,

perf_samples_cf: LedgerColumn<cf::PerfSamples>,

block_height_cf: LedgerColumn<cf::BlockHeight>,

program_costs_cf: LedgerColumn<cf::ProgramCosts>,

bank_hash_cf: LedgerColumn<cf::BankHash>,

optimistic_slots_cf: LedgerColumn<cf::OptimisticSlots>,

merkle_root_meta_cf: LedgerColumn<cf::MerkleRootMeta>,

// 동기화 및 상태 관리

max_root: AtomicU64,

insert_shreds_lock: Mutex<()>,

new_shreds_signals: Mutex<Vec<Sender<bool>>>,

completed_slots_senders: Mutex<Vec<CompletedSlotsSender>>,

lowest_cleanup_slot: RwLock<Slot>,

slots_stats: SlotsStats,

}
```

RocksDB 기반 데이터 저장 구조

Column Family 구조

Blockstore는 다음과 같은 Column Family들로 데이터를 분류하여 저장합니다:

1. 슬롯 메타데이터 관련

- **SlotMeta**: 슬롯의 상태 및 메타데이터
- **DeadSlots**: 죽은 슬롯 표시
- **DuplicateSlots**: 중복 슬롯 증명
- **Orphans**: 부모가 없는 고아 슬롯
- **Root**: 루트로 확정된 슬롯

2. Shred 데이터 관련

- **ShredData**: 데이터 shred 저장
- **ShredCode**: 코딩 shred 저장 (에러 복구용)
- **Index**: Shred 인덱스 정보
- **ErasureMeta**: 에러 복구 메타데이터
- **MerkleRootMeta**: Merkle 트리 루트 메타데이터

3. 트랜잭션 관련

- **TransactionStatus**: 트랜잭션 실행 상태
- **AddressSignatures**: 주소별 서명 인덱스
- **TransactionMemos**: 트랜잭션 메모
- **TransactionStatusIndex**: 트랜잭션 상태 인덱스

4. 블록 정보 관련

- **BankHash**: 뱅크 해시
- **Rewards**: 보상 정보
- **Blocktime**: 블록 타임스탬프
- **BlockHeight**: 블록 높이
- **PerfSamples**: 성능 샘플
- **ProgramCosts**: 프로그램 실행 비용

SlotMeta 구조체

```
pub struct SlotMeta {
pub slot: Slot, // 슬롯 번호
pub consumed: u64, // 연속으로 받은 shred 수
pub received: u64, // 받은 최고 shred 인덱스 + 1
pub first_shred_timestamp: u64, // 첫 shred 수신 시간
pub last_index: Option<u64>, // 마지막 shred 인덱스
pub parent_slot: Option<Slot>, // 부모 슬롯
pub next_slots: Vec<Slot>, // 자식 슬롯들
pub connected_flags: ConnectedFlags, // 연결 상태 플래그
pub completed_data_indexes: BTreeSet<u32>, // 완료된 데이터 인덱스
}
```

주요 기능 구현

1. Blockstore 초기화

```rust
impl Blockstore {
pub fn open(ledger_path: &Path) -> Result<Blockstore> {
Self::do_open(ledger_path, BlockstoreOptions::default())
}

fn do_open(ledger_path: &Path, options: BlockstoreOptions) -> Result<Blockstore
fs::create_dir_all(ledger_path)?;
let blockstore_path = ledger_path.join(
options.column_options.shred_storage_type.blockstore_directory(),
);

// RocksDB 데이터베이스 열기
let db = Database::open(&blockstore_path, options)?;
// Column Family 초기화
let meta_cf = db.column();
let dead_slots_cf = db.column();
// ... 기타 column family들
// 최대 루트 슬롯 찾기
let max_root = db
.iter::<cf::Root>(IteratorMode::End)?
.next()
.map(|(slot, _)| slot)
```

```
.unwrap_or(0);
let blockstore = Blockstore {
ledger_path: ledger_path.to_path_buf(),
db: Arc::new(db),
// ... 필드 초기화
max_root: AtomicU64::new(max_root),
};
blockstore.cleanup_old_entries()?;
blockstore.update_highest_primary_index_slot()?;
Ok(blockstore)
}
}
```

2. Shred 삽입 및 저장

```
impl Blockstore {
pub fn insert_shreds(
&self,
shreds: Vec<Shred>,
leader_schedule: Option<&LeaderScheduleCache>,
is_trusted: bool,
) -> Result<InsertResults> {
let _lock = self.insert_shreds_lock.lock().unwrap();
```

```
// Shred 검증 및 분류
let mut data_shreds = Vec::new();
let mut coding_shreds = Vec::new();
for shred in shreds {
match shred.shred_type() {
ShredType::Data => data_shreds.push(shred),
ShredType::Code => coding_shreds.push(shred),
}
}
// 데이터베이스에 저장
let mut write_batch = self.db.batch()?;
// 데이터 shred 저장
for shred in &data_shreds {
let key = (shred.slot(), u64::from(shred.index()));
write_batch.put::<cf::ShredData>(key, shred.payload())?;
}
// 코딩 shred 저장
for shred in &coding_shreds {
let key = (shred.slot(), u64::from(shred.index()));
write_batch.put::<cf::ShredCode>(key, shred.payload())?;
}
// 배치 커밋
self.db.write(write_batch)?;
```

```
Ok(InsertResults {
completed_data_set_infos: vec![],
duplicate_shreds: vec![],
})
}
}
```

3. 슬롯 메타데이터 관리

```
impl Blockstore {
pub fn meta(&self, slot: Slot) -> Result<Option<SlotMeta>> {
self.meta_cf.get(slot)
}
pub fn is_full(&self, slot: Slot) -> bool {
if let Ok(Some(meta)) = self.meta_cf.get(slot) {
return meta.is_full();
}
false
}
pub fn orphan(&self, slot: Slot) -> Result<Option<bool>> {
self.orphans_cf.get(slot)
}
}
```

4. 에러 복구 메커니즘

```
impl Blockstore {
fn try_shred_recovery(
&self,
erasure_metas: &HashMap<ErasureSetId, WorkingEntry<ErasureMeta>>,
index_working_set: &mut HashMap<u64, IndexMetaWorkingSetEntry>,
prev_inserted_shreds: &HashMap<ShredId, Shred>,
reed_solomon_cache: &ReedSolomonCache,
) -> Vec<Shred> {
let mut recovered_shreds = vec![];
for (erasure_set, working_erasure_meta) in erasure_metas.iter() {
let erasure_meta = working_erasure_meta.as_ref();
let slot = erasure_set.slot();
let index_meta_entry = index_working_set.get_mut(&slot).expect("Index");
let index = &mut index_meta_entry.index;
match erasure_meta.status(index) {
ErasureMetaStatus::CanRecover => {
Self::recover_shreds(
index,
erasure_meta,
prev_inserted_shreds,
&mut recovered_shreds,
```

```
&self.data_shred_cf,
&self.code_shred_cf,
reed_solomon_cache,
);
}
_ => {}
}
}
recovered_shreds
}
}
```

데이터 접근 패턴

1. 슬롯 기반 반복자

```
impl Blockstore {
pub fn slot_meta_iterator(
&self,
slot: Slot,
) -> Result<impl Iterator<Item = (Slot, SlotMeta)> + '_> {
let meta_iter = self.db.iter::<cf::SlotMeta>(
IteratorMode::From(slot, IteratorDirection::Forward)
```

```
)?;
Ok(meta_iter.map(|(slot, slot_meta_bytes)| {
(slot, deserialize(&slot_meta_bytes).unwrap())
}))
}
pub fn rooted_slot_iterator(&self, slot: Slot) -> Result<impl Iterator<Item =
let slot_iterator = self.db.iter::<cf::Root>(
IteratorMode::From(slot, IteratorDirection::Forward)
)?;
Ok(slot_iterator.map(move |(rooted_slot, _)| rooted_slot))
}
}
```

2. Shred 데이터 접근

```
impl Blockstore {
pub fn slot_data_iterator(
&self,
slot: Slot,
index: u64,
) -> Result<impl Iterator<Item = ((u64, u64), Box<[u8]>)> + '_> {
let slot_iterator = self.db.iter::<cf::ShredData>(
IteratorMode::From((slot, index), IteratorDirection::Forward)
```

```
)?;
Ok(slot_iterator.take_while(move |((shred_slot, _), _)| *shred_slot == slot))
}
}
```

성능 최적화

1. 배치 쓰기

Blockstore는 여러 shred를 한 번에 저장하기 위해 RocksDB의 WriteBatch를 사용합니다:

```
let mut write_batch = self.db.batch()?;
// 여러 shred 추가
self.db.write(write_batch)?;
```

2. 컬럼 패밀리 분리

데이터 유형별로 컬럼 패밀리를 분리하여 쿼리 성능을 최적화하고 압축 효율성을 높입니다.

3. 인덱싱

각 슬롯의 shred 존재 여부를 빠르게 확인할 수 있도록 별도의 인덱스를 유지합니다.

데이터 정합성 보장

1. 원자적 업데이트

```
let _lock = self.insert_shreds_lock.lock().unwrap();
```

모든 shred 삽입 작업은 뮤텍스로 보호되어 원자성을 보장합니다.

2. 중복 검사

삽입되는 shred가 이미 존재하는지 확인하고 중복 shred를 처리합니다.

3. 에러 복구

Reed-Solomon 코딩을 사용하여 누락된 데이터 shred를 복구할 수 있습니다.

정리

Blockstore는 Solana의 핵심 저장 시스템으로서 다음과 같은 특징을 가집니다:

1. **RocksDB 기반**: 고성능 키-값 저장소를 활용한 효율적인 데이터 저장
2. **컬럼 패밀리 분리**: 데이터 유형별 최적화된 저장 구조
3. **Shred 기반 저장**: 블록을 작은 조각으로 나누어 네트워크 전송 및 저장 최적화
4. **에러 복구**: Reed-Solomon 코딩을 통한 데이터 복구 기능
5. **원자적 연산**: 데이터 정합성을 보장하는 동시성 제어
6. **효율적인 인덱싱**: 빠른 데이터 검색을 위한 다양한 인덱스 구조

이러한 설계를 통해 Blockstore는 높은 처리량과 안정성을 제공하며, Solana 네트워크의 핵심 인프라 역할을 수행합니다.